

1. Words, tokens, morphology concept questions

Token = one occurrence in text. **Type** = one distinct word (vocabulary entry). "They lay back on the San Francisco grass and looked at the stars and didn't" has 15 tokens but **13 types** (the, and appear twice). 14 if "San Francisco" counts as one word, 16 if "didn't" is split.

Morpheme = smallest meaning-bearing unit: **stem** + **affixes** (star + s). **Inflection** keeps the word class (cat → cats, walk → walking); **derivation** usually changes it (compute → computer → computerize → computerization, clue → clueless).

Evidence for part of speech: **morphological** (takes -ed, -ing, -s ⇒ verb), **distributional** (follows "the", "two" ⇒ noun), semantic.

2. Heaps' Law and Zipf's Law ★ practice Q2

$ V = k \cdot N^\beta$	Heaps' Law: vocabulary size vs corpus size N, with $0 < \beta < 1$. $\beta > 0 \Rightarrow V $ keeps growing without bound ; $\beta < 1 \Rightarrow$ the growth slows (a bit faster than $\sqrt{N}$) but never reaches a ceiling , so unseen words always keep appearing. If $\beta = 0.5$ and the corpus is 4× bigger, V is 2× bigger. (Slides: $\beta \approx 0.6\text{--}0.7$; textbook: $0.44\text{--}0.56$.)
freq $\propto$ 1 / rank	Zipf's Law: the 1000th most common word is about 1/1000 as frequent as the 1st. A few words are very common, most appear once or never ⇒ sparse counts.

3. Byte-pair encoding (BPE) ★ practice Q1

Training: start with the individual characters. Repeat k times: find the **most frequent pair of adjacent symbols**, add the merged pair as a new symbol, replace it everywhere. Every merge adds exactly one entry and removes nothing, so

$ V = V_0 + k$	final vocab = starting symbols + number of merges. Start 9, 8 merges ⇒ 17 . GPT-2: 257 + 50,000 = 50,257.
-------------------	--

Tokenizing new text: split it into characters, then apply the learned merges **in the order they were learned**, greedily, until none apply. Anything can be tokenized (worst case single characters) so there are **no out-of-vocabulary words**. "dog" and "␣dog" (with leading space) are separate entries.

★ **Lecture 3 example**

Corpus set_␣new_␣new_␣renew_␣reset_␣renew; the space is glued to the front of each word. Start symbols {␣ e n r s t w} = **7**. Most frequent pairs: n-e (4), e-w (4), ␣-r (3), r-e (3).
Merges in order: (n,e) (ne,w) (␣,r) (␣r,e) (␣,new) (␣re,new) (s,e) (se,t) ⇒ vocab 7 + 8 = **15**.
Tokenize reset: r e s e t → (s,e) → r e s e t → (se,t) → **[r, e, set]**. A bare r+e was never merged (only ␣r+e), so "re" stays two tokens.
Tokenize set renew → **[set, ␣renew]**.

Tradeoff: smaller vocabulary ⇒ less memory but longer token sequences ⇒ more compute and more tokens billed.

4. N-gram language models ★ practice Q3, Q4

$P(w_1 \dots w_n) = \prod P(w_i w_{i-n+1} \dots w_{i-1})$	Chain rule (product of each word given what came before) plus the Markov assumption : only look back N–1 words. Bigram: $P(\text{transparent} \text{so})$. Trigram: $P(\text{transparent} \text{is, so})$.
$P(w_2 w_1) = C(w_1 w_2) / C(w_1)$	Bigram estimate from counts: "of all the times I saw w_1 , what fraction had w_2 next". Δ Divide by the count of w_1, not by the total number of words .
$P(\text{sentence}) = P(w_1 <s>) \cdot P(w_2 w_1) \cdot \dots \cdot P(</s> w_n)$	Treat <s> and </s> as real tokens : each sentence adds one of each to the counts; include the first word given <s> and </s> given the last word. Δ Forgetting $P(w_i <s>)$ doubles the answer .

★ **Practice quiz corpus (V = 9 types incl. <s>, </s>)**

<s> some are fast </s> <s> and some are slow </s> <s> some are high </s> <s> and some are low </s>

1. $P(\text{some} | \text{and})$: "and" appears 2 times, followed by "some" both times ⇒ $2/2 = \mathbf{1}$.

2. $P(<s> \text{ some are low } </s>) = P(\text{some} | <s>) \times P(\text{are} | \text{some}) \times P(\text{low} | \text{are}) \times P(</s> | \text{low}) = (2/4) \times (4/4) \times (1/4) \times (1/1) = \mathbf{0.125}$.

3. Perplexity of that sentence (4 predicted tokens): $0.125^{-1/4} = 8^{1/4} = \mathbf{1.68}$.

Berkeley Restaurant (V = 1446): $C(I \text{ want}) = 827$, $C(I) = 2533 \Rightarrow P(\text{want} | I) = 0.33$. $P(<s> I \text{ want english food } </s>) = 0.25 \times 0.33 \times 0.0011 \times 0.5 \times 0.68 = 0.000031$.

5. Zeros and smoothing §3.6.1, 3.6.2

The problem: $|V| = 30K \Rightarrow 30K^2 = 900M$ possible bigrams; Shakespeare used only 300K of his 844M ⇒ **99.96% have count 0**. One zero makes the whole sentence probability 0. Smoothing moves a little probability from seen events to unseen ones.

$P = (C(w_1 w_2) + 1) / (C(w_1) + V)$	Add-one (Laplace): pretend every bigram was seen one extra time. Why + V in the bottom: all V possible next words after w_1 got +1, so the total for that row grew by V (keeps the row summing to 1). Δ Adding 1 on top but forgetting V on the bottom .
$P = (C(w_1 w_2) + k) / (C(w_1) + k \cdot V)$	Add-k: same idea with a smaller amount k (0.5, 0.01) so less probability is moved; k is picked on a development set.

★ **Add-one examples**

Quiz corpus, $V = 9$: $P(\text{some} | \text{and}) = (2+1)/(2+9) = 3/11 = \mathbf{0.27}$ (was 1). Unseen $P(\text{low} | \text{and}) = (0+1)/(2+9) = 1/11 = \mathbf{0.09}$ (was 0).
BeRP, $V = 1446$: $P(\text{want} | I) = (827+1)/(2533+1446) = 828/3979 = \mathbf{0.21}$ (was 0.33). Unseen $P(\text{spend} | \text{want}) = 1/(927+1446) = \mathbf{0.00042}$.

6. Perplexity §3.3, lecture 4

$PP(W) = P(W)^{-1/N}$	Perplexity = 1 over the test-set probability, taken to the 1/N power so it is per word. Lower is better . Meaning: the average number of choices the model is "hesitating between" at each word. A model that is always certain has $PP = 1$; a model that guesses uniformly over 30,000 words has $PP = 30,000$.
$H = -(1/N) \sum \log_2 P(w_i \dots)$, $PP = 2^H$	The slides' version: cross-entropy H is the average number of bits per word; perplexity is 2 to that power. Same number as the row above.

WSJ, 38M words train	Unigram	Bigram	Trigram
Perplexity	962	170	109
Entropy (bits)	9.91	7.41	6.77

% reduction bigram → trigram = $(170-109)/170 = \mathbf{36\%}$. Compare only with the **same vocabulary**, test set never seen in training. **Extrinsic** = measure the real application; **intrinsic** = perplexity on held-out text.

7. Logistic regression: score → probability ★ practice Q8

$z = w \cdot x + b$	The score : each feature value × its weight, summed, plus a bias. Can be any real number. x = feature vector of the text (word counts, "has I", log length...).
$\sigma(z) = 1 / (1 + e^{-z})$	Sigmoid squashes the score into a probability $P(y = 1 x)$. $\sigma(0) = 0.5$; large $z \rightarrow 1$; very negative $z \rightarrow 0$. $P(y = 0 x) = 1 - \sigma(z)$. Predict class 1 when $z > 0$.
odds = $p / (1 - p)$	$p = 0.75 \Rightarrow$ odds 3 ("3 to 1"). Ranges 0 to ∞ .
logit = $\ln(p / (1 - p))$	Logit = log of the odds . Ranges $-\infty$ to ∞ , symmetric around 0. $p = 0.75 \Rightarrow \ln \mathbf{3} = \mathbf{1.10}$. $p = 0.5 \Rightarrow 0$. The score z is a logit; solving $z = \ln(p/(1-p))$ for p gives the sigmoid, which is why e appears in it. Δ "3" is the odds, not the logit; $-\ln(0.75)$ is a loss, not a logit .
$P(c x) = e^{z_c} / \sum_j e^{z_j}$	Softmax for k classes: each class has its own weight row (W is $k \times n$), giving k scores; exponentiate each and divide by the sum so they add to 1. Scores [3, 1, 4] $\Rightarrow e^z = [20.1, 2.7, 54.6]$, sum 77.4 $\Rightarrow [0.26, 0.04, \mathbf{0.71}]$.

p	.95	.8	.75	.5	.25	.2	.05
odds	19	4	3	1	1/3	1/4	1/19
logit	2.94	1.39	1.10	0	−1.10	−1.39	−2.94

8. Learning: loss, gradient, update ★ practice Q5, Q6, Q7

$L = -\log \hat{y}$ (if $y = 1$) $L = -\log(1 - \hat{y})$ (if $y = 0$)	Cross-entropy loss = minus the log of the probability the model gave to the correct answer. Model says 0.4, truth is $1 \Rightarrow$ −log 0.4 . Model says 0.3, truth is $0 \Rightarrow$ it gave 0.7 to class 0 $\Rightarrow$ −log 0.7 . Multiclass: −log of the true class's softmax probability. Low probability on the truth ⇒ big loss; perfect $\Rightarrow 0$.
$\partial L / \partial w = (\hat{y} - y) \cdot x$	Gradient = (what the model said − what it should have said) × the input. For softmax, class c uses $(\hat{y}_c - y_c) \cdot x$, with $y_c = 1$ only for the true class. Δ Order is $\hat{y} - y$, not $y - \hat{y}$.
$w \leftarrow w - \eta \cdot \nabla L$	Update: move weights against the gradient, step size η . SGD = one example per update; minibatch = N; batch = whole set.

★ **Practice Q5 (softmax gradient)**

True class James; model output [John 0.2, James 0.4, Alex 0.4]; $x = [-3, 1, 4, 1]$. For James: $(0.4 - 1) \times x = -0.6 \times x = [\mathbf{1.8}, \mathbf{-0.6}, \mathbf{-2.4}, \mathbf{-0.6}]$. For a wrong class like John: $(0.2 - 0) \times x = [-0.6, 0.2, 0.8, 0.2]$.

★ **Lecture 6 gradient step (binary)**

$w = [2.5, -5, -1.2, 0.5, 2.0, 0.7]$, $b = 0.1$, $x = [3, 2, 1, 3, 0, 4.15]$, truth $y = 1$.
1. Score $z = w \cdot x + b = 0.705 + 0.1 = 0.805$; $\hat{y} = \sigma(0.805) = \mathbf{0.69}$.
2. $\hat{y} - y = 0.69 - 1 = -0.31$. Gradient = $-0.31 \times x = [-0.93, -0.62, -0.31, -0.93, 0, -1.29]$.
3. Update with $\eta = 1$: $w -$ gradient = **[3.43, −4.38, −0.89, 1.43, 2.0, 1.98]**. Every weight on a nonzero feature went up, so the score for this example rises toward 1.

9. Precision, recall, F1 §4.9

$P = TP / (TP + FP)$	Of the items I labeled positive, how many really were.
$R = TP / (TP + FN)$	Of the truly positive items, how many I found.
$F1 = 2PR / (P + R)$	Harmonic mean, near the lower of the two: $P = 0.9$, $R = 0.5 \Rightarrow 0.64$. Accuracy misleads with unbalanced classes (TP 10, FP 10, FN 10, TN 970 $\Rightarrow P = R = 0.5$ but accuracy 98%). Macro avg = mean of per-class scores; micro = pool the counts first.

Numbers you may need

$e = 2.72$, $e^2 = 7.39$, $e^3 = 20.1$, $e^4 = 54.6$ · $\ln 2 = 0.69$, $\ln 3 = 1.10$, $\ln 4 = 1.39$ · $\sigma(1) = 0.73$, $\sigma(2) = 0.88$, $\sigma(-z) = 1 - \sigma(z)$ · $-\ln 0.7 = 0.36$, $-\ln 0.5 = 0.69$, $-\ln 0.4 = 0.92$, $-\ln 0.3 = 1.20$.